



Java 사용자 정의 메서드 연습 문제 정답 및 풀이집

본 해설집은 비전공자 교육용으로 작성되었습니다. 각 문제별로 자바의 표준적인 메서드 정의 방식, 접근 제어자(public static), 반환 타입(Return Type), 매개변수(Parameter)의 활용법을 친절한 주석과 함께 풀이합니다.

[기초] 문제 1 ~ 문제 4 풀이

문제 1. 두 수의 합을 반환하는 메서드 (sum)

```
public static int sum(int a, int b) {  
    return a + b; // 두 정수를 더한 결과를 호출한 곳으로 돌려줍니다.  
}
```

💡 비전공자 눈높이 설명:

- **int**: 이 메서드가 실행을 마치고 최종적으로 '정수(int)' 형태의 결과물을 돌려주겠다는 약속(반환 타입)입니다.
- **(int a, int b)**: 메서드가 일할 때 필요한 재료 2개를 각각 정수형 변수 a와 b로 받겠다는 의미입니다.
- **return**: 계산된 결과값을 상자 밖으로 던져서 돌려주는 열쇠입니다.

문제 2. 문자열을 정해진 횟수만큼 반복 출력하는 메서드 (printLoop)

```
public static void printLoop(String str, int n) {  
    for (int i = 0; i < n; i++) {  
        System.out.println(str); // 넘겨받은 문자열(str)을 n번 출력합니다.  
    }  
    // void형 메서드이므로 return을 생략하거나 그냥 'return;'만 쓸 수 있습니다.  
}
```

💡 비전공자 눈높이 설명:

- **void**: "이 메서드는 화면에 출력만 할 뿐, 어떤 값도 다시 돌려주지(반환하지) 않는다"라는 뜻의 특수 타입입니다.
- 화면에 글자를 찍는 단순 노동을 수행할 때는 **void**를 선택합니다.

문제 3. 홀수/짝수를 판별하여 부울(boolean) 값을 반환하는 메서드 (isEven)

```
public static boolean isEven(int number) {  
    if (number % 2 == 0) {  
        return true; // 2로 나눈 나머지가 0이면 짝수이므로 true 반환  
    } else {  
        return false; // 그 외에는 홀수이므로 false 반환  
    }  
}
```

```
}  
}
```

 비전공자 눈높이 설명:

- **boolean** : 참(true) 또는 거짓(false) 둘 중 하나만 반환하는 타입입니다.
- **%** 는 나머지를 구하는 기호입니다. 어떤 수를 2로 나누었을 때 나머지가 0인 성질을 이용해 짝수를 골라냅니다.

문제 4. 세 개의 정수 중 가장 큰 값을 구하는 메서드 (maxOfThree)

```
public static int maxOfThree(int a, int b, int c) {  
    int max = a; // 우선 첫 번째 수(a)가 제일 크다고 가정하고 변수에 담습니다.  
  
    if (b > max) {  
        max = b; // 만약 b가 현재 주인공(max)보다 크다면 주인공을 b로 바꿉니다.  
    }  
    if (c > max) {  
        max = c; // 만약 c가 현재 주인공(max)보다 크다면 주인공을 c로 바꿉니다.  
    }  
  
    return max; // 최종 승리자가 담긴 max를 반환합니다.  
}
```

 비전공자 눈높이 설명:

- 세 개를 동시에 비교하는 것보다, 가상의 1등 상자(**max**)를 만들어 두고 하나씩 토너먼트로 비교하는 것이 컴퓨터가 연산하기 훨씬 쉽고 깔끔한 알고리즘이 됩니다.

[중급] 문제 5 ~ 문제 8 풀이

문제 5. 절대값을 반환하는 메서드 (absoluteValue)

```
public static int absoluteValue(int num) {
    if (num < 0) {
        return -num; // 음수일 경우 마이너스(-)를 한 번 더 붙여서 양수로 만듭니다.
    }
    return num; // 양수나 0이면 받은 그대로 돌려줍니다.
}
```

💡 비전공자 눈높이 설명:

- 수학에서 음수 **-5** 에 마이너스를 한 번 더 붙이면 **-(-5) = +5** 가 되는 원리를 코드로 옮긴 것입니다. **if** 조건이 맞아서 실행되면 아래의 **return num;** 문장에는 도달하지 않고 메서드가 즉시 종료됩니다.

문제 6. 1부터 N까지의 총합을 구하는 메서드 (sumUpTo)

```
public static int sumUpTo(int n) {
    int total = 0; // 합계를 누적해서 더해나갈 통을 준비합니다. (0으로 초기화)

    for (int i = 1; i <= n; i++) {
        total += i; // total = total + i; 와 같은 뜻으로, 계속 숫자를 엮어갑니다.
    }

    return total; // 다 더해진 최종 누적 합계를 돌려줍니다.
}
```

문제 7. 특정 문자열 속에 특정 글자가 몇 개 들어있는지 세는 메서드 (countChar)

```
public static int countChar(String text, char target) {
    int count = 0; // 글자 수를 세기 위한 계시기(카운터)입니다.

    // text.length()는 글자 전체 길이를 알려줍니다.
    for (int i = 0; i < text.length(); i++) {
        // text.charAt(i)는 i번째 방에 있는 글자 한 개를 콧 짚어 꺼내옵니다.
        if (text.charAt(i) == target) {
            count++; // 꺼낸 글자가 내가 찾는 글자(target)와 같다면 카운트 1 증가!
        }
    }
}
```

```
    return count; // 최종 조사 결과를 숫자로 반환합니다.  
}
```

 비전공자 눈높이 설명:

- `text.length()` : 문자열의 글자 길이를 구합니다.
- `text.charAt(i)` : 자바 문자열은 0번째 글자부터 시작합니다. 돋보기를 들고 0번 칸, 1번 칸, 2번 칸을 차례대로 탐색하면서 내가 찾는 알파벳(`target`)이 나올 때마다 카운터를 찰칵찰칵 누르는 동작입니다.

문제 8. 정수 배열을 받아 모든 요소의 평균을 구하는 메서드 (calculateAverage)

```
public static double calculateAverage(int[] arr) {  
    // 만약 들어온 배열이 텅 비어있다면, 0으로 나누는 에러를 막기 위해 안전장치를 둡니다.  
    if (arr.length == 0) {  
        return 0.0;  
    }  
  
    int sum = 0;  
    // 향상된 for문(for-each): 배열 arr에 들어있는 방을 하나씩 순서대로 꺼내 item에 넣어줍니다.  
    for (int item : arr) {  
        sum += item; // 모든 방의 값을 더합니다.  
    }  
  
    // 평균은 소수점이 나올 수 있으므로 형변환((double))을 거쳐 나누어 줍니다.  
    return (double) sum / arr.length;  
}
```

 비전공자 눈높이 설명:

- `int[] arr` : 정수 여러 개를 묶어놓은 '기차 칸(배열)'을 통째로 매개변수로 받겠다는 의미입니다.
- `(double) sum` : 정수끼리 그냥 나누면 자바는 소수점을 다 버리고 몫만 취합니다. 일부러 소수점 계산을 살리기 위해 마법을 걸어 `double` (실수형)로 변신시켜 나눗셈을 수행합니다.

[고급] 문제 9 ~ 문제 10 풀이

문제 9. 팩토리얼(Factorial, $N!$)을 구하는 메서드 (factorial)

```
public static long factorial(int n) {
    long result = 1; // 곱셈을 누적할 때는 0이 아니라 1로 시작해야 합니다.

    for (int i = 1; i <= n; i++) {
        result *= i; // result = result * i; 1부터 n까지 차례대로 곱합니다.
    }

    return result;
}
```

💡 비전공자 눈높이 설명:

- 팩토리얼은 숫자가 조금만 커져도 계산 결과가 상상을 초월할 정도로 엄청나게 커집니다(예: 20! 등). 따라서 일반 정수형 상자인 `int` 대신 훨씬 거대한 숫자를 담을 수 있는 대형 상자인 `long` 타입을 반환형으로 사용한 것입니다.

문제 10. 소수(Prime Number) 여부를 판별하는 메서드 (isPrime)

```
public static boolean isPrime(int num) {
    // 1보다 작거나 같은 수(0, 1, 음수 등)는 수학적으로 소수가 아닙니다.
    if (num <= 1) {
        return false;
    }

    // 2부터 시작해서 내 직전 숫자(num - 1)까지 나를 나누어 떨어뜨리는 녀석이 있는지 검사합니다.
    for (int i = 2; i < num; i++) {
        if (num % i == 0) {
            return false; // 나 외에 누군가에게 딱 나누어 떨어졌다면? 소수가 아니므로 탈락(false)!
        }
    }

    // 위 고난의 테스트(for문)를 한 번도 안 걸리고 무사히 통과했다면 소수가 확실합니다!
    return true;
}
```

💡 비전공자 눈높이 설명:

- 소수는 '1과 자기 자신으로만 나누어떨어지는 1보다 큰 자연수'입니다.

- 예컨대 `num` 이 7이라면, 2부터 6까지의 숫자로 7을 계속 나누어 봅니다. 나머지가 0이 되는 숫자가 하나도 없으므로 `for` 문을 무사히 통과하여 최종적으로 `true` (소수가 맞음)를 돌려주게 됩니다.



메서드 연습 문제를 테스트하는 통합 실행 클래스 예시

본인이 작성한 연습용 메서드들이 정말 잘 작동하는지 `main` 메서드에서 아래와 같이 감싸서 실행해 볼 수 있습니다.

```
public class MethodPractice {
    public static void main(String[] args) {
        // 문제 1 테스트
        System.out.println("3 + 5 = " + sum(3, 5));

        // 문제 3 테스트
        System.out.println("7은 짝수인가요? " + isEven(7));

        // 문제 10 테스트
        System.out.println("11은 소수인가요? " + isPrime(11));
    }

    // 여기에 작성하신 사용자 정의 메서드들을 public static ... 으로 붙여넣으면 됩니다.
}
```